Software Requirements Specification

Hansraj Query Bridge *(including the HansBot conversational assistant)*

Version 1.0 Prepared by: **The Null Pointers** Smart India Hackathon 2026 For: Hansraj College, University of Delhi

How to use this document: Every place a diagram would normally appear in an SRS has been replaced with a boxed 🖼 **Diagram Instruction**. It tells you the diagram type, what to put in it, and how to lay it out, so you (or a teammate) can draw it in draw.io / Figma / Lucidchart / PowerPoint and drop the image in before final submission.

1. Introduction

1.1 Purpose

The purpose of this document is to present a detailed description of the **Hansraj Query Bridge**, a web-based query-resolution and ticketing platform for Hansraj College. It explains the purpose and features of the system, the interfaces of the system, what the system will do, the constraints under which it must operate, and how the system will react to external stimuli. This document is intended for both the stakeholders (college administration, the Teacher-in-Charge body, and students) and the developers of the system, and is submitted in partial fulfilment of Smart India Hackathon 2026 by team **The Null Pointers**.

1.2 Scope

The Hansraj Query Bridge is a role-based web platform that lets a **Student** raise an administrative or academic query, get it answered instantly by an AI assistant (**HansBot**) if the answer is already verified/known, or have it automatically escalated into a tracked ticket that is routed to the correct department and resolved by a **Teacher-in-Charge (TIC)** or **Admin**.

The system is designed to:

Reduce repeated in-person visits to college offices for routine questions (certificates, fees, scholarships, timetables, subject changes, etc.).
Give students a single place to track the status of every query they've raised, instead of chasing answers over email, WhatsApp, or physical office windows.
Give staff a structured queue instead of scattered emails, with document requests, resolution notes, and an auditable status history per ticket.
Continuously grow a verified knowledge base, so that once a question is answered by a human once, HansBot can answer it instantly for the next student who asks something similar.

Out of scope for this version: real-time voice support, payment processing (fee payment itself continues on the existing DU/college portal — this system only answers *questions about* fees), and mobile native apps (the web app is responsive but there is no separate iOS/Android build).

1.3 Definitions, Acronyms, and Abbreviations

Term	Definition
Student	Any user who submits queries seeking resolution; the primary end user of the Student Portal.
TIC (Teacher-in-Charge)	Faculty member responsible for reviewing and resolving queries routed to their department.
Admin	College office staff with system-wide visibility — can view, reassign, and manage all queries, departments, and users.
HansBot	The AI chatbot component that answers queries instantly using semantic search over a verified knowledge base, or escalates them into a ticket.
Query / Ticket	A single student question and its full lifecycle record (status, assigned department, uploaded documents, resolution).
Knowledge Base (KB)	The store of previously verified question–answer pairs that HansBot searches to answer instantly.
Semantic Search	Search based on the *meaning* of a query rather than exact keyword matching, using vector embeddings.
RBAC	Role-Based Access Control — restricting what a logged-in user can see/do based on their assigned role (Student / TIC / Admin).
Authorized Email Database	A stored table of pre-approved college email IDs; login is only granted to an email present in this table (see §3.1.4).
API	Application Programming Interface.
UI	User Interface.

1.4 References

IEEE Std 830-1998, *IEEE Recommended Practice for Software Requirements Specifications*, IEEE Computer Society, 1998.
Smart India Hackathon 2026 — Problem Statement (Hansraj College query-resolution system).
Hansraj Query Bridge architecture notes and UI mockups (team-internal, referenced throughout §3).

1.5 Overview

Section 2 (**Overall Description**) gives an informal, non-technical overview of the product — what it does, who uses it, and under what constraints. Section 3 (**Specific Requirements**) is written for

developers and testers and gives the precise, testable requirements the system must satisfy. Both sections describe the same product but for different audiences.

2. Overall Description

2.1 Product Perspective

The Hansraj Query Bridge is a new, self-contained system — it is not a modification of an existing product. It replaces the informal, ad-hoc process of students emailing or physically visiting departments with queries. It has three client-facing surfaces (Student Portal, TIC Dashboard, Admin Dashboard) sitting on top of one shared backend, an AI/semantic-search engine, and a relational database.

It has one external dependency worth noting for interface purposes: **Google Sign-In**, used purely to authenticate the user's identity (see §3.1.4) — the actual authorization decision (is this person allowed in?) is made by the platform's own stored **Authorized Email Database**, not by Google.

🖼 **Diagram Instruction — Figure 1: System Environment / Architecture Diagram** Draw a **layered box diagram**, top to bottom:

Top layer ("Client Layer"): three boxes side by side — *Student Portal*, *TIC Dashboard*, *Admin Dashboard*. Label the layer.
Second layer ("Gateway & API Layer"): one box — *API Gateway* (handles auth + routing) — with arrows coming down from all three client boxes into it, labeled `REST / HTTPS`.
Third layer ("Core Services"): five or six smaller boxes side by side — *HansBot / Semantic Search Engine*, *Query & Ticket Service*, *Department Routing Service*, *Document Upload Handler*, *Notification Engine* — each connected up to the Gateway box with double-headed arrows.
Bottom layer ("Data Layer"): two cylinder shapes (standard database symbol) — *Relational Database (Postgres)* and *Vector/Knowledge Base Store* — connected up to the Core Services layer.
Off to the side, draw a small separate box labeled **Google Sign-In (external)**, connected only to the API Gateway with a dotted line labeled `Identity verification only`. This is the single most important diagram in the document — every reviewer will look at it first, so keep the boxes evenly sized and the arrows uncluttered.

2.2 Product Functions

At a high level, the system performs the following functions:

Authenticate a user via Google Sign-In and check their email against the Authorized Email Database, then load their role.
Answer student questions instantly through HansBot's semantic search against the verified knowledge base.
Escalate unanswered or low-confidence questions into a formal, tracked ticket.
Route each ticket to the correct department automatically.
Track every ticket through a defined status lifecycle (New → In Progress → Waiting for Documents → Resolved).

Collect supporting documents from students when a staff member requests them.
Notify students by email whenever their ticket's status changes.
Grow the knowledge base automatically whenever a staff member marks a resolved ticket as reusable.
Give staff and admins dashboards to view, filter, resolve, and (for admins) reassign tickets across the whole college.

2.3 User Characteristics

Student: Assumed to be comfortable using a modern web browser and typing natural-language questions (in English or common Hinglish phrasing). No technical background assumed.
Teacher-in-Charge (TIC): Assumed to be comfortable with basic office software (email, simple web forms); not assumed to have any technical background. Uses the dashboard periodically, not continuously.
Admin: Assumed to have slightly higher familiarity with dashboards/filters, since they manage cross-department data and occasionally reassign misrouted tickets.

2.4 General Constraints

The system must run entirely as a web application accessible over standard college/home internet connections — no dedicated institutional network required.
Login is restricted to college email addresses that exist in the Authorized Email Database (see §3.1.4); the system does not support open self-registration.
The system must degrade gracefully — if the AI/semantic-search component is unavailable, every query must still be escalated into a normal ticket rather than the platform failing outright.
Document uploads are capped in size and restricted to common document formats (PDF/JPEG/PNG) for storage-cost and security reasons.

2.5 Assumptions and Dependencies

It is assumed the college will provide/maintain the list of valid staff and student email addresses that populate the Authorized Email Database.
It is assumed Google Sign-In remains available as an identity provider; if Google's service is down, login is unavailable college-wide (this is treated as an acceptable, disclosed risk rather than something this system re-engineers around).
It is assumed the existing college systems (admissions, fee portal, examination cell) are **not** integrated directly — HansBot answers *about* these processes but does not read live data from them in this version.

3. Specific Requirements

3.1 External Interface Requirements

3.1.1 User Interfaces

The system exposes three distinct web interfaces, all reachable from the same responsive site:

Public Homepage — reachable without login. Shows what the system is for, a rotating "what can you ask" FAQ panel with sample questions, and three login entry points (Student / TIC / Admin).
Student Portal — after login, shows the HansBot chat window and a "My Queries" list with each query's live status and any action buttons (Upload Documents / See Solution).
TIC Dashboard — after login, shows a queue of tickets routed to that TIC's department, each with the ability to open, request documents, and mark resolved.
Admin Dashboard — after login, shows all tickets college-wide, with filters by department/status, and the ability to reassign a misrouted ticket.

🖼 **Diagram Instruction — Figure 2: Screen Flow / Navigation Diagram** Draw a simple **flowchart of screens** (not code logic — just which screen leads to which):

Start box: *Homepage*.
From Homepage, three arrows to three boxes: *Login (Student)*, *Login (TIC)*, *Login (Admin)*.
Each login box has one arrow leading to its respective dashboard box: *Student Dashboard*, *TIC Dashboard*, *Admin Dashboard*.
From *Student Dashboard*, branch into three boxes: *HansBot Chat*, *My Queries List*, *Raise New Query Form* — and from *My Queries List*, branch into *Upload Documents* and *View Resolution*.
From *TIC Dashboard*, branch into *Ticket Detail View* → *Request Documents* and *Resolve Ticket*.
From *Admin Dashboard*, branch into *All Tickets View* → *Reassign Ticket* and *Manage Users/Departments*.
Use plain rectangles for screens and arrows for navigation; this is meant to be readable by a non-technical evaluator at a glance.

3.1.2 Hardware Interfaces

None specific — the system requires only a standard client device (laptop, desktop, or smartphone) with a modern web browser and an internet connection. No specialized hardware, sensors, or peripherals are required.

3.1.3 Software Interfaces

Google Identity Services — used only to verify that the person signing in genuinely controls the email address they claim (an OAuth 2.0 identity check). It does **not** decide who is allowed into the system.
Relational Database — stores users, the Authorized Email Database, tickets, departments, and documents metadata.
Vector/Semantic Search Store — stores knowledge-base entries as embeddings for HansBot's similarity search.
Object Storage — stores uploaded documents (certificates, receipts, etc.) outside the main database.
Transactional Email Service — sends status-change notification emails to students.

3.1.4 Communication Interfaces

All client–server communication happens over standard HTTPS. The chat interface additionally uses a streaming connection so HansBot's replies appear progressively rather than all at once.

Login/authentication flow (updated):

The user clicks "Login" and completes Google Sign-In, proving they control a given email address. The system takes that verified email address and checks it against a **stored Authorized Email Database** — a table containing **only** the email IDs (`email_id`) of people permitted to use the system, each tagged with their role (Student / TIC / Admin).
If the email exists in this table, login succeeds and the user is placed into their recorded role; if it does not exist, login is rejected with a clear "this email is not registered for Hansraj Query Bridge" message.
No password, OTP, or additional credential is stored or checked by this system — Google Sign-In establishes *identity*, and the Authorized Email Database establishes *authorization*.

🖼 **Diagram Instruction — Figure 3: Login/Authentication Sequence Diagram** Draw a **UML-style sequence diagram** with three vertical lifelines: *User (Browser)*, *Hansraj Query Bridge Server*, *Google Sign-In*, and *Authorized Email Database*.

Arrow from User → Google Sign-In: "Sign in with Google".
Arrow back Google Sign-In → User: "Verified identity token (email)".
Arrow User → Server: "Send token".
Arrow Server → Authorized Email Database: "Look up email_id".
Two possible return arrows from the database, drawn as a branch: (a) "Found — return role" → Server → User: "Login success, redirect to dashboard"; (b) "Not found" → Server → User: "Login rejected". Keep it to these five/six steps — this diagram exists to prove you understand your own auth flow, not to show every internal function call.

3.2 Functional Requirements

Functional requirements are grouped by **Mode**, matching each role's dashboard.

3.2.1 Mode 1 — Student Mode

3.2.1.1 Login *Description:* A student authenticates via Google Sign-In; the system checks the email against the Authorized Email Database (§3.1.4) and, if found with role = Student, grants access to the Student Portal. *Input:* Google-verified email. *Processing:* Lookup in Authorized Email Database; load stored role and profile. *Output:* Redirect to Student Dashboard, or a rejection message.

3.2.1.2 Ask HansBot *Description:* The student types a natural-language question into the HansBot chat window. *Input:* Free-text query. *Processing:* The system embeds the query, searches the knowledge base for a semantically similar, previously verified answer. If a sufficiently confident match is found, it is returned instantly. Otherwise, HansBot drafts a clearly worded version of the question and asks the student whether to submit it as a formal query. *Output:* Either an instant verified answer, or a proposed ticket the student can confirm.

3.2.1.3 Raise a New Query *Description:* The student explicitly submits a query (either directly via a "Raise a New Query" button, or by confirming HansBot's proposed escalation). *Input:* Query text; optional supporting attachment. *Processing:* A new ticket is created with status `Pending`, and the Department Routing Service assigns it to a department. *Output:* New entry appears in the student's "My Queries" list.

3.2.1.4 View My Queries *Description:* The student sees a list of every query they have raised, each with its current status. *Input:* None (student is already logged in). *Processing:* Retrieve all tickets

belonging to the logged-in student, ordered by most recent. *Output:* A list showing, for each ticket, its title, date raised, and a colour-coded status (Pending / In Progress / Documents Needed / Resolved) with a relevant action button.

3.2.1.5 Upload Requested Documents *Description:* When a ticket's status is "Documents Needed", the student can upload the requested file(s). *Input:* One or more files (PDF/JPEG/PNG). *Processing:* System validates file type and size, stores the file, links it to the ticket, and updates status back to "In Progress". *Output:* Confirmation of successful upload; ticket status updates.

3.2.1.6 View Resolution *Description:* When a ticket is marked "Resolved", the student can view the staff-written solution. *Input:* Student selects "See Solution" on a resolved ticket. *Processing:* Retrieve and display the resolution text stored against that ticket. *Output:* Resolution text shown to the student.

3.2.2 Mode 2 — Teacher-in-Charge (TIC) Mode

3.2.2.1 Login *Description:* Same mechanism as §3.2.1.1, but the Authorized Email Database entry must have role = TIC.

3.2.2.2 View Assigned Queries *Description:* The TIC sees every ticket routed to their department, with status and date. *Processing:* Retrieve tickets filtered by `department_id` matching the TIC's assigned department. *Output:* A queue/list view, sortable by status or date.

3.2.2.3 Request Documents *Description:* The TIC can ask the student to upload supporting documents before the query can be resolved. *Processing:* Ticket status is changed to "Documents Needed"; a notification email is queued for the student. *Output:* Student sees the updated status and an upload prompt.

3.2.2.4 Resolve Query *Description:* The TIC writes a solution and marks the ticket resolved. *Input:* Free-text resolution. *Processing:* Ticket status updated to "Resolved"; resolution text stored; notification email queued. *Output:* Student can now view the resolution (§3.2.1.6).

3.2.2.5 Add Resolution to Knowledge Base *Description:* The TIC can flag a resolved ticket's question–answer pair as reusable. *Processing:* The Q&A pair is embedded and stored in the knowledge base so HansBot can answer similar future questions instantly. *Output:* The pair becomes searchable by HansBot immediately.

3.2.3 Mode 3 — Admin Mode

3.2.3.1 Login *Description:* Same mechanism as §3.2.1.1, but the Authorized Email Database entry must have role = Admin.

3.2.3.2 View All Queries *Description:* The Admin sees every ticket across every department, with filters by department, status, and date range.

3.2.3.3 Reassign a Misrouted Query *Description:* If the automatic Department Routing Service assigns a ticket incorrectly, the Admin can manually move it to the correct department. *Processing:* Ticket's `department_id` is updated; the newly assigned TIC(s) are notified.

3.2.3.4 Manage Users and Departments *Description:* The Admin can add/remove entries in the Authorized Email Database (i.e. approve a new staff email, revoke a departed staff member's

access) and manage the list of departments used for routing.

3.3 Performance Requirements

HansBot's semantic search must return an instant answer (or the escalation prompt) within **2 seconds** under normal load for a single query.
The Student, TIC, and Admin dashboards must load a ticket list of up to 100 items within **3 seconds**.
The system should comfortably support the full expected concurrent user base of the college (several thousand students, tens of staff) during peak periods such as admission/exam season without visible slowdown.

3.4 Design Constraints

Authentication must go through Google Sign-In for identity, combined with the platform's own Authorized Email Database for authorization — no separate password system is to be built.
All uploaded documents must be stored outside the main relational database (object storage), with only a reference/key stored in the database, to keep the database lean and backups fast.
The knowledge base and the relational data (users, tickets, departments) should live in the same database engine (using a vector-search-capable extension) to avoid keeping two separate databases in sync.

🖼 **Diagram Instruction — Figure 4: Logical Data Model / Entity-Relationship (ER) Diagram** Draw a standard **ER diagram** with the following entities as boxes, each listing its key fields, connected by relationship lines:

User: `email_id (key)`, `role`, `department_id (nullable)`, `name`.
Department: `department_id (key)`, `name`.
Ticket: `ticket_id (key)`, `student_email_id`, `department_id`, `title`, `description`, `status`, `date_raised`.
Document: `document_id (key)`, `ticket_id`, `file_url`, `uploaded_at`.
Resolution: `ticket_id (key)`, `resolved_by_email_id`, `resolution_text`, `resolved_at`.
KnowledgeBaseEntry: `kb_id (key)`, `question_text`, `answer_text`, `embedding_vector`, `added_from_ticket_id`. Draw lines: User (Student) `1–many` Ticket; Department `1–many` Ticket; Department `1–many` User (for TIC role); Ticket `1–many` Document; Ticket `1–1` Resolution; Resolution `1–0/1` KnowledgeBaseEntry (dashed line, since not every resolution is added to the KB). Use crow's-foot or simple "1" / "N" labels on each line end — whichever notation your course expects.

3.5 Attributes

(Software quality attributes — availability, security, maintainability, portability.)

Security: Access is restricted at login by the Authorized Email Database (§3.1.4); once logged in, Role-Based Access Control (RBAC) ensures a Student can only ever see their own tickets, a TIC only their department's tickets, and only an Admin can see and reassign everything. Uploaded documents are stored privately and only accessible via time-limited links to the relevant TIC/Admin.
Availability: The core ticketing flow (raise → route → resolve → notify) must keep working even if the AI/semantic-search component is temporarily unavailable — in that case, every query is simply escalated straight to a ticket instead of attempting an instant answer.

Maintainability: The system is organized into clearly separated services (auth, tickets, routing, documents, notifications, AI) so that any one part can be updated or replaced without touching the others.
Portability: Being a standard responsive web application, the system runs on any modern browser on desktop or mobile without a separate native app build.

3.6 Other Requirements

Data retention: Resolved tickets and their resolutions should be retained for at least one academic year for audit purposes.
Notifications: Every status change on a student's ticket (Documents Needed, Resolved) must trigger an email notification, so a student is never required to keep the tab open and manually refresh to know their query was answered.
Knowledge base quality: Only a TIC or Admin can add a resolution to the knowledge base — students cannot self-publish answers, to avoid unverified information being served by HansBot.

Summary of Diagrams to Prepare

#	Diagram	Where it belongs	Type
1	System Environment / Architecture	§2.1	Layered box diagram
2	Screen Flow / Navigation	§3.1.1	Flowchart
3	Login/Authentication Sequence	§3.1.4	UML sequence diagram
4	Logical Data Model	§3.4	ER diagram

(Optional, if your evaluator expects UML formally: you can additionally draw one Use Case Diagram with three actor stick-figures — Student, TIC, Admin — each connected to their ovals from §3.2.1–3.2.3. This is a nice-to-have summary diagram, not a strict requirement of the IEEE 830 structure used here.)